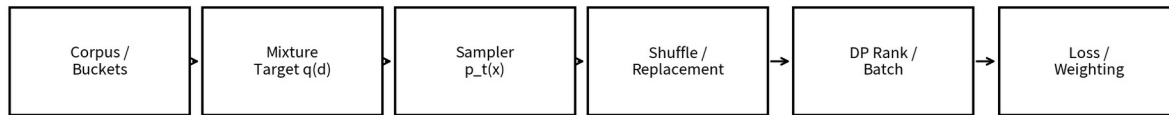


LLM 预训练之 Sampling

训练采样 / Data Sampling 深度研究手册

从静态语料配方到每一步训练轨迹：随机性、重采样、排序、重复、在线自适应与分布式可复现

LLM 预训练 Sampling：把静态数据集变成训练轨迹



Sampling 的工程对象不是“一个百分比”，而是训练时随 step / epoch / model state 变化的随机过程

核心视角：Sampling 是一个随 step / epoch / rank / model state 变化的随机过程；它决定“谁在什么时候贡献梯度”

Research cut-off: 2026-08-13 | 01-B-Agent

执行摘要：Sampling 决定训练轨迹，而不只是数据比例

预训练语料即使完全相同，只要改变样本访问顺序、replacement、每个数据源的实时采样概率、每个 batch 的组成、重复次数或在线权重，最终梯度序列就会改变。Sampling 因此处在 Data Mixture 与 Optimizer 之间：它把“想训练什么分布”变成“每一步实际看到什么”。

核心判断	Know-why	工程含义
Mixture $\neq$ Sampling	Mixture 是长期目标比例；Sampler 决定瞬时轨迹	必须同时记录 target weights 与 realized exposure
Uniform 并不等于中性	均匀按文档、按 token、按 shard 的结果完全不同	先定义 sampling unit，再讨论“uniform”
Shuffle 是优化变量	相邻样本相关会改变梯度噪声与收敛	shuffle quality 要成为可观测指标
重复不是绝对有害	前几次重复可能价值接近 unique data，之后边际效用下降	需要 repetition budget / per-doc count
Static 不一定最优	样本价值会随模型训练进度变化	online sampling / reweighting 可形成反馈闭环
分布式正确性优先	多 rank 重复或漏样会悄悄改变训练分布	global order、rank partition、resume 都需可复现

本册最重要的 Know-why Sampling 决定的是梯度估计器。数据层真正进入优化器的不是“语料库本身”，而是由采样概率 $p_t(x)$ 、batch 组成和损失权重共同定义的随机梯度。

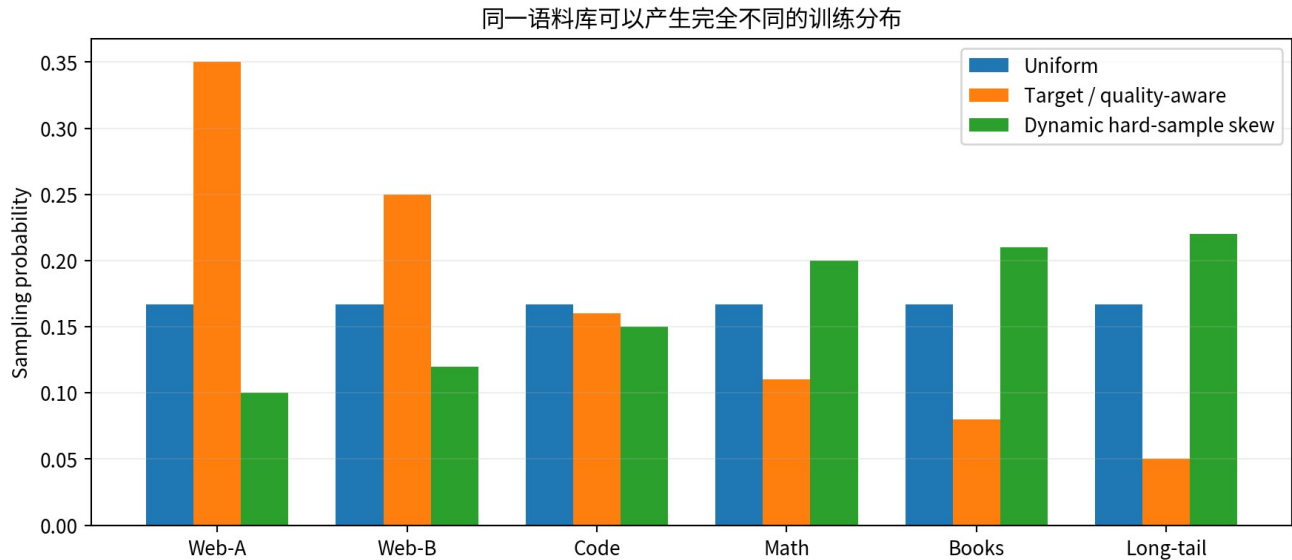


图 1 同一数据池可以通过不同采样策略形成完全不同的训练分布。柱高为机制示意。

0. Sampling、Data Mixture、Sequence Construction 的边界

层	回答的问题	典型变量	输出
Data Mixture	长期希望各域占多少？	domain weights / temperature / caps	目标分布 $q(d)$
Sampling	这一 step / epoch 实际取谁？	$p_t(x)$ 、shuffle、replacement、curriculum	sample IDs / batch composition
Sequence Construction	取出的文档如何变成 token sequence？	packing、EOS、mask、position	tokens / labels / masks
Optimization	样本梯度怎么组合？	loss weight、batch average、optimizer	parameter update

工程中最常见的误区，是把 mixture weight 当作已经实现的训练分布。实际上，shard size、epoch_size、with-replacement、drop_last、distributed partition、packing 和 failure/retry 都可能让 realized exposure 偏离配置值。

1. 数学视角：Sampler 定义随机梯度估计器

设我们真正希望优化的目标分布为 $q(x)$ ，但为了效率或课程学习，从 $p_t(x)$ 采样。若直接平均梯度，模型实际优化的是 p_t 对应的经验目标；若希望保持 q 的无偏目标，可使用 importance weight $w_t(x)=q(x)/p_t(x)$ 。

$$g_t = (1/B) \sum \nabla \ell(x_i; \theta_t), x_i \sim p_t(x); \text{若重加权, 则 } g_t = (1/B) \sum [q(x_i)/p_t(x_i)] \nabla \ell(x_i; \theta_t).$$

设计	收益	风险
改变 p_t ，不做 loss correction	真正改变训练目标，常用于 curriculum / quality sampling	可能偏离原目标分布
改变 p_t + importance weight	理论上可校正为目标分布	权重方差大时梯度噪声上升
固定 p_t ，改变 loss weight	实现简单，可在线 per-sample reweight	统计语义与 sampling 不完全相同
先离线筛选再 uniform	运行时最简单	无法适应 model state；多样性永久丢失

关键概念 Sampling 不是数据 I/O 的附属功能，而是优化目标与梯度方差的组成部分。讨论 sampling 时必须同时说明：sampling unit、probability、replacement、loss correction、epoch definition。

2. Uniform Random Sampling：强 baseline，但先定义“uniform”

所谓 Uniform	实际概率	隐含偏差
Uniform document	每篇文档等概率	短文档与长文档贡献相同抽样机会，但 token 数不同
Uniform token	每个 token 等概率	长文档更容易被抽到
Uniform sequence	构造后的 sequence 等概率	受 packing / truncation 强烈影响
Uniform shard	每个 shard 等概率	不同 shard 大小时会扭曲 sample exposure
Uniform domain	每个 domain 等概率	小域会被强烈 upsample

Megatron Core 的 GPTDataset 使用 document index、sample index 和 shuffle index，并按随机种子构造确定性文档/样本顺序；BlendedDataset 则按权重从多个 dataset 中抽样，直到达到目标 composite size。官方文档明确把数据 blend 与 sample lookup 分成不同索引层。[1]

Know-how：把“uniform”写进配置时，最好同时写成 uniform_over=document / token / sequence / domain，而不要只保留一个布尔参数。

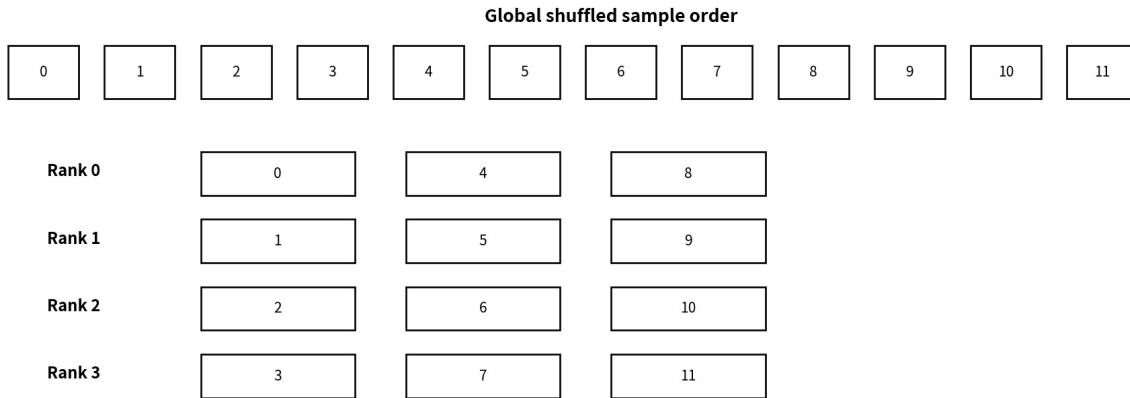
3. Shuffle：不是美化随机性，而是控制梯度相关性

MosaicML Streaming 文档明确指出，未充分 shuffle 的数据可能导致 loss spike、训练发散或次优收敛；但完全全局 shuffle 会造成严重跨节点下载与 I/O 成本，因此大规模系统通常采用 shard-level + block-level + intra-node 的分层随机化。[2]

Shuffle 粒度	随机性	I/O / 内存	典型风险
Global permutation	最高	最贵	大规模远程 shard 下载失控
Shard order only	低-中	低	同 shard 内样本仍高度相关
Block shuffle	中-高	可控	block 太小则相关性残留
Intra-shard	中	最低	跨 shard 结构仍可能周期化
Pre-shuffled + light runtime shuffle	高（依赖离线质量）	低	数据版本变化需重建

PyTorch DistributedSampler 还要求每个 epoch 调用 `set_epoch()`，否则跨 epoch 会重复使用同一随机顺序；这类细节不会报错，却会悄悄改变训练轨迹。[3]

Distributed Sampling：随机性、去重叠与可复现必须同时成立



目标：每个 data-parallel rank 得到不重叠子序列，同时所有 rank 对同一个 seed/epoch 具有一致、可恢复的全局定义

图 2 分布式采样不仅要“随机”，还必须保证不同 data-parallel rank 的样本分片不重叠且可复现。

4. With Replacement vs Without Replacement

方式	定义	优势	风险 / 适用
Without replacement	一个 epoch 内每样本最多一次	覆盖率清晰、重复可控；工程中最常见	数据流需可枚举；epoch 边界明显
With replacement	每次独立按 $p(x)$ 抽样	天然支持任意权重、无限流和动态 p_t	同一阶段可频繁重复，coverage 不稳定
Weighted without replacement	按权重排序/抽取但限制重复	兼顾偏好与覆盖	算法与分布式实现更复杂
Reservoir / streaming sampling	在线处理未知长度流	适合持续爬取和无限数据	严格全局权重与恢复更难

统计学里 without-replacement 随机 reshuffling 与经典独立同分布采样并不完全等价；实践中它常因覆盖稳定、实现简单而表现良好。对于 LLM 预训练，更关键的是“是否重复”与“重复多少次”通常由 epoch / total token budget / dataset size 共同决定。

5. 重复采样：Repetition Budget 与 Token Crisis

Scaling Data-Constrained Language Models 在最多 900B training tokens、9B 参数的实验中发现：固定 compute 下，把有限数据重复到约 4 epochs，loss 与拥有同量 unique data 相比变化很小；但继续重复时，新增 compute 的边际价值最终衰减。[4]

2025 年 NeurIPS 的 Datasets, Documents, and Repetitions 又展示了更细的结论：在适当训练 recipe 下，强过滤/去重后的高质量数据重复最高约 10 epochs，仍可优于一个 10 倍更大的 superset 单次训练；同时文档级重复次数不应一视同仁。[5]

重复采样不是二元问题：边际价值取决于数据质量与训练 recipe

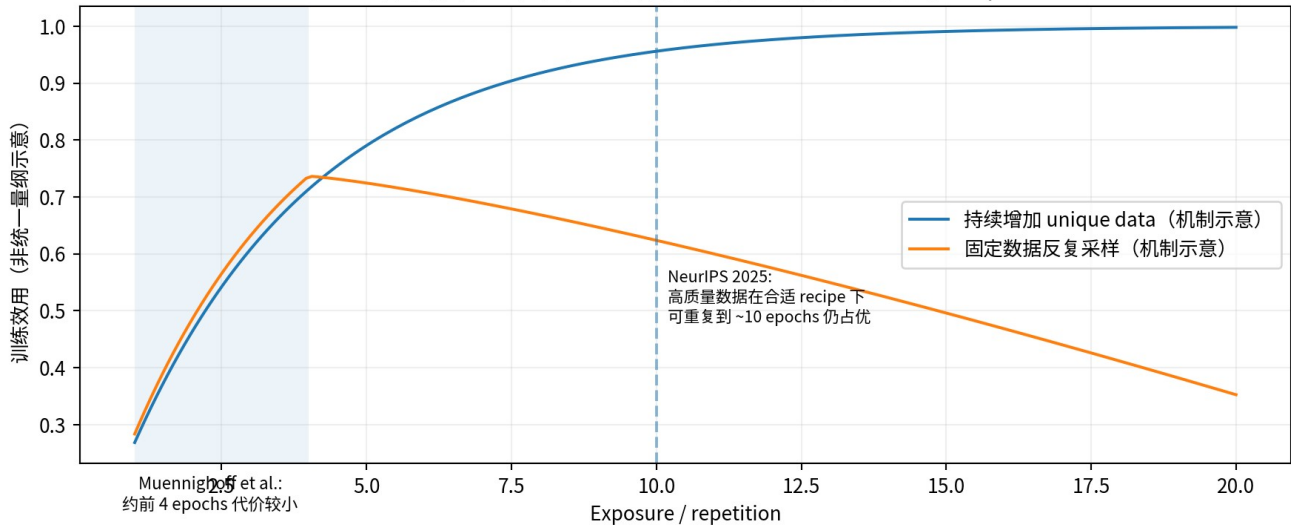


图3 重复的边际价值取决于数据质量、模型规模、训练目标与 recipe；曲线本身为机制示意，阴影/标注来自论文结论。

应该监控	定义	为什么重要
Exposure count	每文档/每 cluster 被训练多少次	避免少数文档无限被复用
Unique-token ratio	累计 unique tokens / train tokens	衡量真实新增信息
Repeat histogram	$P(\text{exposure}=k)$	观察长尾重复是否失控
Loss vs exposure	同类数据重复后 loss 的衰减	判断何时已经“学会”
Memorization probes	高重复文档的逐字生成/隐私风险	重复不仅影响泛化，也影响记忆

6. Importance Resampling：从“随机抽”到“匹配目标分布”

DSIR (Data Selection with Importance Resampling) 把原始数据与目标数据投影到 hashed n-gram 特征空间，估计密度比并按 importance weight 重采样。NeurIPS 2023 报告其可在 4.5 小时从完整 The Pile 中选择 100M documents；hashed n-gram 上的 KL reduction 与平均 downstream accuracy 相关系数 $r=0.82$ ，并在 general-domain 设置对 GLUE 相比 random / heuristic baseline 提升约 2-2.5%。[6]

DSIR 步骤	Know-how	Know-why
定义 target sample	用目标领域少量无标签样本	明确“像谁”比抽象质量更可操作
低维特征化	hashed n-grams	避免对 web-scale 文档做昂贵 embedding / LM scoring
估计 density ratio	$q(z)/p(z)$	把数据选择转成概率重采样
importance resample	按权重选择 subset / stream	让 selected data 更接近 target
KL reduction 检查	比较 selection 前后 target proximity	在真正预训练前提供低成本 proxy

边界 DSIR 更像 target-distribution matching；它不是通用“质量评分器”。如果 target 选择本身窄或有偏，importance sampling 会非常忠实地放大这种目标。

7. Instance-level Online Reweighting：样本价值随模型变化

2025 年 ICLR 的 Dynamic Loss-Based Sample Reweighting 指出，LLM 预训练常在巨大语料上近似一次性看到样本，传统依赖样本历史统计的 curriculum 不容易扩展。该工作提出完全在线的 per-sample loss reweighting，并系统研究不同 loss

区间的权重；其结论之一是下调低-loss（已容易/可能冗余）样本往往更有效，并在 124M 到 7B 模型实验中展示更快收敛和更好性能。[7]

信号	直觉	常见危险
High loss	可能是难样本 / 新知识，也可能是噪声	盲目追 high-loss 会追垃圾数据
Medium loss	通常仍可学习且不是极端异常	定义依赖模型阶段和 domain
Low loss	可能已学会/高度重复	也可能是基础语法，不应全部丢弃
Loss change	衡量 learnability / progress	需要至少短期状态或重复观测
Gradient influence	更接近“对目标的真实贡献”	计算成本高，需代理模型

Know-how: online sample reweighting 和 sampling 可以组合。前者改变 batch 内每条样本的梯度系数，后者改变它出现的概率；两者同时使用时必须记录 effective weight，避免重复放大。

8. Model-aware Selection : MATES 与 Group-MATES

MATES (NeurIPS 2024) 通过局部 probing 获得“oracle data influence”，再训练一个小 data influence model，预测当前训练阶段最有价值的数据。论文在 C4 上预训练 410M 与 1B 模型时，相比随机选择显著改善，并报告达到某些性能所需总 FLOPs 可减半。[8]

Group-MATES (NeurIPS 2025) 进一步指出样本价值并非独立可加：数据之间存在冗余、互补和覆盖关系，因此在 cluster / group 层建模 relational influence。其在 DCLM 400M/1B/3B 设置报告相对随机选择 3.5%-9.4% 的 downstream 相对增益，并将达到某些性能所需 token 数最多降低约 1.75x。[9]

方法	粒度	是否动态	额外成本	解决的问题
Uniform	sample	否	极低	强 baseline
DSIR	sample / feature	离线	低-中	目标分布匹配
MATES	sample	阶段性动态	中	模型当前最需要什么
Group-MATES	group / cluster	阶段性动态	中-高	样本之间的冗余与互补
Loss reweight	sample	每 step	极低	当前难度/冗余

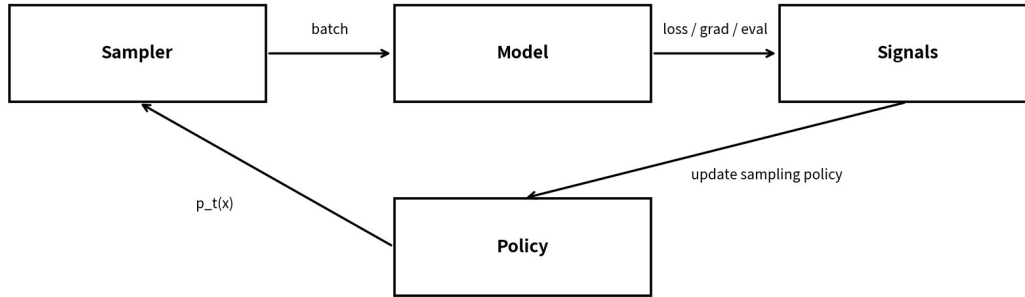
9. Online Sampling : ODM / ADO 把 mixture 变成训练反馈闭环

Online Data Mixing (ODM) 使用 multi-armed bandit 在训练过程中动态调整数据组采样概率；论文报告在其设置中达到 next-best 方法最终 perplexity 所需迭代减少 19%，5-shot MMLU 相对 accuracy 提升 1.9%，且额外 wall-clock 开销很小。[10]

Adaptive Data Optimization (ADO, ICLR 2025) 进一步用每个 domain 的在线 scaling-law 估计“剩余 learning potential”，不依赖额外 proxy model，也不修改模型 update，而是并行更新数据分布。其目标是把 sampling policy 本身变成训练中的低开销控制器。[11]

PDS (ICLR 2025) 则把 data selection 写成 optimal control，并用 Pontryagin Maximum Principle 近似最优选择条件；论文报告在有限数据情形可把数据需求降低约 1.8x，并用 scaling-law 外推讨论到约 400B 模型、10T tokens 的尺度。外推结果与直接大规模实证证据应区分解读。[12]

Online Sampling：模型状态反过来决定下一阶段看什么数据

图 4 Online Sampling 的核心：模型产生反馈信号，Sampler 根据当前训练状态更新 $p_t(x)$ 。

10. Curriculum：Data Order 本身就是训练算法

2025 的 Beyond Random Sampling 对 LLM 预训练 curriculum 做系统实验，比较 vanilla curriculum、pacing-based sampling 和 interleaved curriculum，并用 compression ratio、lexical diversity、readability 等难度指标排序。论文报告 curriculum 在早中期更快收敛，作为 warmup 时最终 benchmark 最多约 +3.5%。[13]

2026 的 Curriculum Learning for LLM Pretraining 进一步分析学习动力学：多个 curriculum 并未创造完全不同的“学习阶段”，更像是在相同宏观阶段中降低梯度噪声、改变数据暴露；随着模型规模增大，curriculum 的收益可能缩小。[14]

Curriculum 轴	例子	风险
Difficulty	easy → hard / interleave	“难度”指标本身可能偏向某语言或文体
Quality	high-quality warmup / anneal	过度集中可能损失多样性
Length	short → long	会与 sequence length / batch size 联动
Domain	general → code/math / continual PT	容易产生遗忘或过拟合目标域
Freshness	old → recent / recency boosts	会引入时间偏差与知识漂移

Know-why Random shuffle 假设“顺序不重要”；curriculum 的核心反驳是：有限训练预算下，参数路径依赖使“先学什么、后学什么”可以改变梯度噪声、收敛速度与最终解。

11. Batch Composition：Sampler 最终输出的不是样本，而是梯度集合

Batching 策略	特点	适用
Random batch	按全局 sample order 顺序组 batch	默认 baseline；domain 比例仅长期接近期望
Stratified batch	每 batch 强制包含若干域/质量桶	降低短期比例方差，适合小 batch / 多域
Per-stream batch	一个 batch 只来自一个 stream	便于 domain-specific loss / routing；梯度方差更大
Device-per-stream	不同 device 固定不同 stream	便于流水线或专门 kernel，但同步梯度语义复杂
Difficulty-balanced	按 loss / difficulty 分桶后组合	控制 hard/easy 比例，需在线信号

MosaicML Streaming 当前 API 显式支持 random、stratified、per_stream 与 device_per_stream batching_method；这说明 batch composition 是 sampler 的一等工程变量，而不只是 DataLoader 末端的实现细节。[15]

如果 global batch 很大，长期 mixture 的瞬时波动通常会被平均；但当 batch 变小、domain 极不平衡、或使用高方差 importance weights 时，stratified batch 可以显著降低“某一步完全没有关键域”的概率。

12. Distributed Sampling：正确性、性能、可恢复三者必须同时成立

Megatron Core 的大规模数据加载设计会预先构造并缓存 index mapping，所有 rank 使用相同随机定义，再由 sampler 让 data-parallel ranks 获得 disjoint subset；其官方文档强调 shared read-only indices、无跨 rank contention、确定性 permutation 与 data-parallel sharding。[1]

必须保证	失败症状	检查方法
Rank disjointness	不同 GPU 重复训练同一样本	记录 global sample id / overlap rate
Global coverage	某些样本永远没被访问	epoch coverage / unseen count
Seed + epoch determinism	resume 后顺序漂移	checkpoint sampler state / hash
Worker determinism	worker 数变化导致次序变化	canonical partition / worker-independent ID mapping
drop_last / pad semantics	尾 batch 重复或丢失	记录 dropped / padded sample count
Elastic resume	扩缩容后重复/跳过	全局 sample cursor 而非本地 iterator state

PyTorch DistributedSampler 在 drop_last=False 时会补额外 indices 使各 replica 可整除，这可能产生尾部重复；在严格统计场景需要明确是否允许这种 padding。[3]

13. Streaming Sampling：随机性与 I/O 局部性之间的系统 trade-off

MosaicML Streaming 将 dataset 先按 canonical nodes 划分，再对 shard order 与 node 内 sample IDs 做随机化，从而减少不同训练节点重复下载 shard；其 naive global shuffle 随机性最高，却可能让所有节点下载几乎所有 shards，严重拖慢吞吐。[2]

系统变量	调大后的效果	代价
shuffle block size	更接近全局随机	需要更大 cache / 更多 shard 活跃
predownload	隐藏远程 I/O latency	本地缓存压力上升
canonical nodes	增加可恢复的独立 sample 路径	配置变化更复杂
epoch_size	控制每 epoch 实际 exposure	“epoch”不再等于完整 dataset pass
sampling_method balanced/fixed	balanced 每 epoch 重新抽；fixed 重复同一 subset	决定跨 epoch coverage

工程原则 大规模预训练的“最好 shuffle”通常不是统计意义上的完全随机 permutation，而是满足训练质量阈值的最低 I/O 成本随机化。应该同时监控 shuffle quality 与 tokens/sec，而不是只看其中一个。

14. Sampling Observability：配置值不等于 realized distribution

指标	推荐粒度	含义
Realized source share	step / 1B tokens / entire run	实际各域 token 比例
Exposure histogram	document / cluster	每条数据被看过多少次
Coverage	dataset / domain	在给定 token budget 下覆盖多少 unique items

指标	推荐粒度	含义
Shuffle locality	adjacent source/domain/shard correlation	发现随机性不足
Batch entropy	per batch	batch 是否被单一域主导
Importance ESS	per step/window	importance weight 是否导致有效样本数骤降
Sampling lag	online policy	policy 更新到数据真正到 GPU 的延迟
Rank overlap	distributed	不同 DP rank 是否重复样本
Resume delta	checkpoint boundary	恢复前后是否有重复/跳过

对 importance weights w_i ，可用 $ESS \approx (\sum w_i)^2 / \sum w_i^2$ 观察有效 batch size。如果名义 batch 很大但少数样本权重极端，真实梯度统计可能仍像小 batch。

15. 如何做 Sampling Ablation：固定住真正应该固定的东西

必须固定	原因
Raw corpus + cleaning + dedup	否则无法归因给 sampling
Tokenizer + Sequence Builder	它们会改变 sample/token 粒度与 loss targets
Total train tokens / FLOPs	避免“某方法只是多看了数据”
Model / optimizer / LR / batch token budget	隔离 sampler 对梯度轨迹的影响
Eval suite + checkpoint cadence	观察短期收敛和最终能力
Random seeds（至少多个）	sampling 本身方差很大，单 seed 易误判

实验轴	baseline	variant	看什么
Replacement	random reshuffle	with replacement	coverage / loss / repeats
Shuffle quality	block	global-ish / intra-shard	loss spikes / throughput
Repetition	1 epoch	2/4/8/10 epochs	val loss / memorization
Granularity	domain-level	instance/group-level	downstream / overhead
Static vs online	fixed $p(x)$	ODM / ADO / loss-based	learning curve / stability
Batching	random	stratified	gradient variance / domain volatility

不要只比较最终 accuracy Sampling 方法最常见的价值是更快到达同一性能，因此至少要画 performance-vs-tokens、performance-vs-FLOPs、performance-vs-wall-clock 三条曲线。

16. Production Sampling Blueprint

推荐把 sampler 设计成一个显式、可版本化、可恢复的“训练数据控制平面”，而不是 DataLoader 里的隐式随机函数。

层	职责	应持久化的状态
Policy layer	target mixture / curriculum / online weights	policy version、weights、update step
Selection layer	sample / group / stream 抽样	sample IDs、probabilities、exposure counters
Ordering layer	shuffle / replacement / epoch cursor	seed、epoch、global cursor
Partition layer	DP rank / worker 分片	world size、canonical partition、rank mapping
Batch layer	random / stratified / per-stream	batch recipe、effective weights
Sequence handoff	交给 Sequence Builder	document IDs、boundary metadata
Observability	realized distribution / overlap / ESS	windowed metrics + audit logs

一个稳健的训练 checkpoint 不应只包含 model/optimizer/LR scheduler；如果希望严格可重复 resume，还应保存 sampler 的 global progress、epoch/seed、动态 policy 状态、stream cursor 与必要的 exposure state。

17. 方法选择决策树

你的问题	优先方法	原因
语料足够大，只求稳定 baseline	random reshuffle + distributed disjoint sampling	最少假设、最易复现
想让 corpus 更像某目标域	DSIR / importance resampling	显式做 target matching
域比例可能随训练阶段改变	ODM / ADO / dynamic mixture	在线调整 group probability
想做细粒度 data efficiency	loss-based / MATES / Group-MATES	利用 instance/group utility
高质量数据有限，需要重复	repetition budget + exposure observability	把重复当连续变量而非禁忌
远程流式数据吞吐瓶颈	block/shard-aware shuffle	平衡随机性与 I/O locality
batch 很小且 domain 极不均衡	stratified batching	降低瞬时 domain variance
resume / elastic scale 频繁	global-ID deterministic sampler	避免重复/跳样

18. Know-Why / Know-How 最终沉淀

- Know-why 1: 预训练数据真正进入优化器的是“随机梯度分布”，不是静态 corpus。
- Know-why 2: Mixture 决定长期配方，Sampling 决定训练路径；路径依赖使二者不能互换。
- Know-why 3: Shuffle 的目标不是数学上最随机，而是在足够低的样本相关性与系统吞吐之间做 Pareto 优化。
- Know-why 4: 重复的边际价值会下降，但下降速度取决于质量、模型规模、数据大小与 recipe；“绝不重复”并不是普适规律。
- Know-why 5: 样本 utility 是动态的。早期有用的数据在后期可能已学会，在线 sampling 因此有理论与工程动机。
- Know-how 1: 所有 sampler 配置必须明确 sampling unit、replacement、probability、loss correction、epoch semantics。
- Know-how 2: 记录 realized exposure，而不是相信配置的 mixture weights；至少按 100M/1B token window 做审计。
- Know-how 3: 分布式 sampler 必须测试 rank overlap、coverage、set_epoch、drop_last/padding 和 resume delta。
- Know-how 4: 重要性采样要监控权重方差与 ESS；极端权重可能把大 batch 退化成少量样本主导。
- Know-how 5: 动态方法必须把 policy update latency、额外 FLOPs 与 wall-clock 一起计入，不能只比较 benchmark。
- Know-how 6: Sampling ablation 要固定 corpus、tokenizer、sequence construction、train tokens、optimizer，并做多 seed。

一句话总结 Sampling = “谁在什么时候贡献多少梯度”。当预训练进入高质量数据有限、长 horizon、在线数据与大规模分布式时代，Sampler 已经从 DataLoader 的配角变成训练算法本身。

参考资料与证据等级

优先列出论文正式会议页、作者/机构页面与官方框架文档。文中数值只在原论文实验范围内解读；2026 年尚未正式发表的预印本不作为核心结论唯一依据。

- [1] NVIDIA Megatron Core - Datasets / Data Loading at Scale — 官方工程文档. [source](#)
- [2] MosaicML Streaming - Shuffling — 官方工程文档. [source](#)
- [3] PyTorch DistributedSampler documentation — 官方工程文档. [source](#)
- [4] Muennighoff et al. Scaling Data-Constrained Language Models, NeurIPS 2023 — 正式论文. [source](#)
- [5] Fang et al. Datasets, Documents, and Repetitions, NeurIPS 2025 — 正式论文. [source](#)
- [6] Xie et al. Data Selection via Importance Resampling (DSIR), NeurIPS 2023 — 正式论文. [source](#)
- [7] Sow et al. Dynamic Loss-Based Sample Reweighting, ICLR 2025 — 正式/会议版本. [source](#)
- [8] Yu et al. MATES, NeurIPS 2024 — 正式论文. [source](#)
- [9] Yu et al. Group-Level Data Selection / Group-MATES, NeurIPS 2025 — 正式论文. [source](#)
- [10] Albalak et al. Efficient Online Data Mixing (ODM), 2023 — 论文/预印本. [source](#)
- [11] Jiang et al. Adaptive Data Optimization (ADO), ICLR 2025 — 正式论文. [source](#)
- [12] Gu et al. Data Selection via Optimal Control (PDS), ICLR 2025 — 正式论文. [source](#)
- [13] Zhang et al. Beyond Random Sampling: Curriculum Learning, 2025 — 预印本. [source](#)
- [14] Elgaar & Amiri. Curriculum Learning for LLM Pretraining, 2026 — 预印本. [source](#)
- [15] MosaicML StreamingDataset API - sampling/batching methods — 官方工程文档. [source](#)
- [16] Tirumala et al. D4: De-duplication and Diversification, NeurIPS 2023 — 正式论文. [source](#)
- [17] Xue et al. To Repeat or Not To Repeat, NeurIPS 2023 — 正式论文. [source](#)
- [18] Shamir. Without-Replacement Sampling for Stochastic Gradient Methods, 2016 — 理论论文. [source](#)

研究结论的证据强弱

证据等级	本册如何使用
A - 正式会议/官方实现	作为核心工程结论与数值依据
B - 作者/机构页面或成熟预印本	用于补充方法细节或最新方向
C - 2026 新预印本/未充分复现方向	只作为前沿观察，不作为生产默认